

MAXSQUASH

Club Details & Member Booking API Changes

Version 1.4

Project	MaxSquash
Prepared by	Usama
Audience	Backend Team

Define the required player-facing API fields and the revised court-booking behavior for authenticated club members.

1. Scope Summary

- Update the player-facing APIs so each club object includes complete club information, non-member booking rules, allowed booking time windows, and the authenticated player membership status.
- Apply the same club data fields in player/dashboard, player/clubs, and player/clubs/{clubId}.
- When an authenticated player is an approved member of the selected club, court booking must skip payment and directly reserve the selected slot.
- Non-members must continue through the club non-member booking policy and the existing payment flow where booking is allowed.

Implementation Principle

Do not create separate club-policy calls for these screens. The required booking and membership information must be included directly in the existing player APIs.

2. Affected Player APIs

Method	Endpoint	Required Change
GET	player/dashboard	All club objects on the dashboard must include the required additional fields.
GET	player/clubs	Each club in the list must include booking policy and authenticated-player membership information.
GET	player/clubs/{clubId}	Include complete club details, booking rules, membership state, and derived booking/payment permissions.

3. Required API Changes

Add the fields listed below to the existing club objects in player/dashboard, player/clubs, and player/clubs/{clubId}. Do not change unrelated fields, pagination, authentication, or the current API flow.

Field	Type	Required Behavior
id	integer	Club identifier.
name	string	Club display name.
logo / image	string null	Public club image URL.
address	string null	Club address or location.
contact details	object null	Existing public contact information where applicable.

allow_non_member_booking	boolean	Whether a player who is not an approved club member may book a court.
non_member_booking_start_time	string null	Daily start time for non-member booking. Null when non-member booking is disabled or no restricted window exists.
non_member_booking_end_time	string null	Daily end time for non-member booking. Null when non-member booking is disabled or no restricted window exists.
is_member	boolean	True only when the authenticated player has an approved active membership in this club.
membership_status	string null	approved, pending, rejected, removed, or null when no membership record exists.
membership_number	string null	Authenticated player membership number for this club, when available.
can_book	boolean	Derived backend flag showing whether the authenticated player can currently proceed to slot selection/booking.
requires_payment	boolean	False for approved members. For non-members, follow the current paid-booking rules.

4. Endpoint-Specific Requirements

4.1 GET player/dashboard

- Add the required club policy and authenticated-player membership fields to each club object on the dashboard.
- Membership and payment flags must be calculated specifically for the authenticated player, not used as generic club defaults.

4.2 GET player/clubs

- Add the required fields to every club object in the club list.
- Existing search, filters, sorting, and pagination may remain unchanged.
- The API must avoid N+1 membership queries by loading the authenticated player memberships efficiently.

4.3 GET player/clubs/{clubId}

- Keep all current club-detail fields unchanged.
- Add non-member booking policy, allowed time window, is_member, membership_status, membership_number, can_book, and requires_payment to the club details.
- If the club does not exist or is inactive, use the project-standard not-found handling.

5. Court Booking Flow

Player Condition	Booking Permission	Payment Behavior
Approved member of the club	May book according to normal court and slot availability.	No payment. Reserve the slot directly.
Pending membership	Treat as non-member until approved.	Apply non-member policy and payment flow.
Rejected / removed membership	Treat as non-member.	Apply non-member policy and payment flow.
No membership; non-member booking enabled	Only within the club-defined non-member time window.	Use the existing payment flow.
No membership; non-member booking disabled	Booking must be blocked.	No payment attempt should be created.

Critical Rule

An approved club member must never be redirected to payment for a court booking at that club. The booking endpoint must verify membership on the server and directly reserve the slot.

6. Backend Booking Validation

1. Identify the authenticated player from the access token.
2. Load the selected club and confirm that it is active.
3. Check whether the player has an active approved membership for the selected club.

4. If approved, set `requires_payment` = false and continue directly to slot reservation.
5. If not approved, validate `allow_non_member_booking` and the configured booking time window.
6. For an eligible non-member, continue with the existing payment-required booking process.
7. Validate slot availability again inside a database transaction before confirming the booking.
8. Prevent duplicate or concurrent reservations for the same court and time slot.

7. Validation and Error Scenarios

Scenario	Expected Result
Non-member booking is disabled	Use HTTP 403 with a clear club-members-only message.
Non-member requests a slot outside allowed hours	Use HTTP 422 or the project-standard validation handling.
Member record exists but is not approved	Do not grant the free member-booking benefit.
Selected slot is no longer available	Use the project-standard conflict handling and do not create payment or booking duplication.
Club or court is inactive	Block booking using the existing standardized error format.

8. Acceptance Criteria

- All three player APIs return consistent club booking-policy and authenticated-membership fields.
- `is_member` is true only for an active approved membership.
- Approved members can book courts without payment.
- Non-members can book only when the club permits it and only during the configured time window.
- Non-members who are allowed to book continue through the current payment flow.
- Booking authorization is validated on the backend and cannot be bypassed by client-provided flags.
- Existing pagination, authentication, and unrelated fields remain unchanged.

Final Note

The mobile application may use `can_book` and `requires_payment` for navigation, but the booking endpoint remains the source of truth and must repeat all permission checks.